

Start coding or [generate](#) with AI.

Lab — MNIST Classifier: Target 97%+

Build, train, and evaluate a neural network on the full MNIST dataset. Visualise predictions, inspect the confusion matrix, examine misclassified digits, and experiment with different architectures to understand how design choices affect accuracy.

Complete Lab Code

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import confusion_matrix, classification_report

# Load and preprocess MNIST
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
X_train = X_train.reshape(-1, 784).astype('float32') / 255.0
X_test = X_test.reshape(-1, 784).astype('float32') / 255.0
# Visualise a sample of training digits
fig, axes = plt.subplots(2, 10, figsize=(14, 3))
for d in range(10):
    for r, offset in enumerate([0, 1]):
        idx = np.where(y_train == d)[0][offset]
        axes[r, d].imshow(X_train[idx].reshape(28,28), cmap='gray')
        axes[r, d].set_title(str(d), fontsize=8); axes[r, d].axis('off')
plt.suptitle('MNIST: 2 samples per digit'); plt.tight_layout(); plt.show()
# Build model as a reusable function
def make_model(h1=256, h2=128, drop=0.3):
    m = keras.Sequential([
        keras.Input(shape=(784,)),
        layers.Dense(h1, activation='relu'),
        layers.Dropout(drop),
        layers.Dense(h2, activation='relu'),
        layers.Dropout(drop),
        layers.Dense(10, activation='softmax'),
    ])
    m.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m
model = make_model()
model.summary()
```

[Show hidden output](#)

```
# Train
history = model.fit(
    X_train, y_train, epochs=30, batch_size=128,
    validation_split=0.1,
    callbacks=[EarlyStopping(patience=5, restore_best_weights=True)],
    verbose=1)
# Evaluate
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Accuracy: {acc*100:.2f}%')
# Confusion matrix
y_pred = np.argmax(model.predict(X_test), axis=1)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(9,7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=range(10), yticklabels=range(10))
plt.title(f'Confusion Matrix - Test Accuracy {acc*100:.1f}%')
plt.xlabel('Predicted'); plt.ylabel('True')
plt.tight_layout(); plt.show()
# Diagonal = correctly classified. Off-diagonal = confused pairs.
# Common confusions: 4 vs 9, 3 vs 8, 5 vs 6
print(classification_report(y_test, y_pred))
# Show misclassified images
wrong = np.where(y_pred != y_test)[0]
print(f'Misclassified: {len(wrong)} out of {len(y_test)}')
fig, axes = plt.subplots(3, 8, figsize=(14, 6))
for i, ax in enumerate(axes.flat):
    if i < len(wrong):
        idx = wrong[i]
```

```

ax.imshow(X_test[idx].reshape(28,28), cmap='gray')
ax.set_title(f'T:{y_test[idx]} P:{y_pred[idx]}',
            color='red', fontsize=7)
ax.axis('off')
plt.suptitle('Misclassified Digits (T=True, P=Predicted)')
plt.tight_layout(); plt.show()
# ■ Architecture comparison ■
configs = [
(64, 32, 0.0, 'Small / no dropout'),
(128, 64, 0.2, 'Medium / light dropout'),
(256, 128, 0.3, 'Large / moderate dropout'),
]
for h1, h2, dr, label in configs:
    m = make_model(h1, h2, dr)
    m.fit(X_train, y_train, epochs=15, batch_size=128,
        validation_split=0.1, verbose=0,
        callbacks=[EarlyStopping(patience=3, restore_best_weights=True)])
    a = m.evaluate(X_test, y_test, verbose=0)
    print(f'{label:35s} Accuracy: {a*100:.2f}%')

```

[Show hidden output](#)

Bonus: Visualise What Neurons Learn

```

# ■ Each Layer 1 neuron has 784 weights (one per pixel) ■
# Reshape to 28x28 to see which pixel patterns each neuron responds to
weights = model.layers[0].get_weights()[0] # shape: (784, 256)
fig, axes = plt.subplots(4, 8, figsize=(14, 7))
for i, ax in enumerate(axes.flat):
    w = weights[:, i].reshape(28, 28)
    ax.imshow(w, cmap='RdBu', vmin=-w.max(), vmax=w.max())
    ax.axis('off')
plt.suptitle('Layer 1 Neuron Weight Patterns — What Each Neuron Detects')
plt.tight_layout(); plt.show()
# Blue = neuron activated strongly by bright pixel here
# Red = neuron suppressed by bright pixel here

```

Layer 1 Neuron Weight Patterns — What Each Neuron Detects



